Enums and Matching

CS-3180

Types

- Rust has an incredibly rich type system
- The most of any language, I've ever used
- We've talked about simple types
- And how their ownership is enforced via the *borrow-checker*
- We'll explore how to make custom types

Enums

- *Enumerations*: language feature that represent a set of fixed values
- Generally, compilers have the ability to "see" all of the values
- We'll call these different values *variants*
- Come in different favors
- We'll look at C -> Java -> Rust

Simple Enums

- C-style enums are as simple as they come
- Alias for integers types
- Finite-set of values

```c
 // start at 0, count up
typedef enum {
    CLUBS,    // = 0
    DIAMONDS, // = 1
    HEARTS,   // ...
    SPADES
} CardSuit;
```

```c
CardSuit mySuit = HEARTS; // int mySuit = 2
CardSuit otherSuit = CLUBS; // int otherSuit = 0;

if (mySuit == HEARTS) {
    printf("You drew Hearts!\n");
}
```

Why does this help?

- Consider the alternatives:
- Just use integers -> Impacts readability
- Four booleans? -> State becomes hard to manage
- Additional slice of type-safety
- Encode state as a type instead

Type-System Alignment

- C is a very *weakly* typed language
- Based on what we know (basically everything is just a int)
- Logical implementation for C
- Enums usually built to fit the existing type ecosystem
- (Often a product of them being added later)

Java Enums

- Enums are often thought of as typed-strings
- Consider how Java handles Enums

```
1 public enum Day {
2     SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
3     THURSDAY, FRIDAY, SATURDAY
4 }
```

```
1 Day today = Day.FRIDAY;
2 Day tmr = Day.SATURDAY;
3 if (today == Day.FRIDAY) {
4     System.out.println("It's Friday!");
5 }
```

Java Enums Alignment

- To fit within the ecosystem, they compile into special classes
- Gives you methods like `ordinal()`,`values()`, `name()`
- Beyond that, they *are* classes
- Meaning you can add any class-like behavior to them
- Enables really rich types
- Let's see that:

Enums as Classes

```
 1 enum Connection {
 2     DISCONNECTED,
 3     CONNECTING,
 4     CONNECTED;
 5
 6     public boolean isReady() {
 7         return this == CONNECTED;
 8     }
 9
10     @Override
11     public String toString(){
12         return "ConnectionType: " + name();
13     }
14 }
15
16 public class Driver {
```

Enums as Classes

```
1  enum Connection {
2      DISCONNECTED,
3      CONNECTING,
4      CONNECTED;
5
6      public boolean isReady() {
7          return this == CONNECTED;
8      }
9
10     @Override
11     public String toString(){
12         return "ConnectionType: " + name();
13     }
14 }
15
16 public class Driver {
```

Enums as Classes

```
    DISCONNECTED,
    CONNECTING,
    CONNECTED;

    public boolean isReady() {
        return this == CONNECTED;
    }

    @Override
    public String toString(){
        return "ConnectionType: " + name();
    }
}

public class Driver {
    public static void main(String[] args) {
        Connection c = Connection.DISCONNECTED;
        c = Connection.CONNECTING;
```

Enums as Classes

```
12          return "ConnectionType: " + name();
13      }
14  }
15
16  public class Driver {
17      public static void main(String[] args) {
18          Connection c = Connection.DISCONNECTED;
19          c = Connection.CONNECTING;
20
21          if(!c.isReady()){
22              c = Connection.CONNECTED;
23          }
24
25          System.out.println(c);
26      }
27  }
```

Enums as Classes

```
 1 enum Connection {
 2     DISCONNECTED,
 3     CONNECTING,
 4     CONNECTED;
 5
 6     public boolean isReady() {
 7         return this == CONNECTED;
 8     }
 9
10     @Override
11     public String toString(){
12         return "ConnectionType: " + name();
13     }
14 }
15
16 public class Driver {
```

Rust Enums

- Same concept as before
- Define with `enum` keyword

```
1 enum Coin {
2     Penny,
3     Nickel,
4     Dime,
5     Quarter
6 }
```

```
 1 fn main() {
 2     let c1: Coin = Coin::Penny;
 3     let c2: Coin = Coin::Dime;
 4     // Rust's list type is Vec
 5     // use vec![...] macro to create a vector of values
 6     let change: Vec<Coin> = vec![
 7         c1,
 8         c2
 9     ];
10 }
```

Enums as Types

- We can pass these into functions
- But ownership rules apply

```
1  fn trade(coin: Coin) -> Coin {
2      println!("You gave me a coin");
3
4      // this is a bad trade
5      Coin::Penny
6  }
7
8  fn main() {
9      let c1: Coin = Coin::Quarter;
10
11     let new_coin = trade(c1);
12     // c1 is no longer valid
13 }
```

Aside: Derive Debug

- Unable to print enums by default
- Need to add `#[derive(Debug)]` trait
- Print with `{:?}`

```
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter
}
```

```
fn trade(coin: Coin) -> Coin {
    // need to use {:?} for debug printing
    println!("You gave me a {:?}", coin);

    // this is a bad trade
    Coin::Penny
}
```

Aside: Derive Debug

- Unable to print enums by default
- Need to add `#[derive(Debug)]` trait
- Print with `{:?}`
- Details later, but this actually a type of *Reflection*

```
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter
}
```

```
fn trade(coin: Coin) -> Coin {
    // need to use {:?} for debug printing
    println!("You gave me a {:?}", coin);

    // this is a bad trade
    Coin::Penny
}
```

Aside: Derive PartialEq

- You'll likely also want `#[derive(PartialEq)]`
- Allows use of `==`

```
#[derive(Debug, PartialEq)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter
}
```

```
fn main() {
    let c1 = Coin::Dime;

    if c1 == Coin::Dime {
        println!("You have 10 cents!");
    }
}
```

Pattern Matching

- Fancy form of conditionals
- Usually evaluate to a type
- Work like a switch case
- Evaluate structure *and* values
- Many languages have a similar feature
- But we'll focus on Rust for now

Match

- `match` keyword and `=>`
- Rust's `match` is *exhaustive* handle every case
- Expression -> Evaluates to a value
- Comma separate options, can use {}s for blocks

```
let boolean = false;
let binary = match boolean {
    false => 0,
    true => {
        println!("Matched one!");
        1
    }
};
```

Rich Matching

- `|` for logical OR
- `..` for range
- `_` for default

```
let number = 13;

println!("Tell me about {}", number);
match number {
    1 => println!("One!"),
    2 | 3 | 5 | 7 | 11 => println!("This is a prime"),
    13..=19 => println!("Teenage"),
    _ => println!("Ain't special"),
}
```

Matching Enums

- You can `match` on enum variants
- (More common than using `==`)

```
fn get_value(coin: &Coin) -> i32 {
    match coin {
        Coin::Penny => 1,
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter => 25
    }
}

fn main() {
    let c1: Coin = Coin::Quarter;
    println!("{}", get_value(&c1));
}
```

Inner Values

- Variants can also hold data (of whatever type)
- (Retrieving this data is trickier, details later)

```
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(String) // state of quarter
}

fn main() {
    let c1= Coin::Quarter(
        String::from("Ohio")
    );
    let c2 = Coin::Penny;
}
```

Enum Composition

- Better use, use another enum!

```
enum State {
    Alabama,
    Alaska,
    // ...
    Wyoming
}
// all sub enums must also derive debug
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(State)
}

fn main() {
```

Enum Composition

- Better use, use another enum!

```
enum State {
    Alabama,
    Alaska,
    // ...
    Wyoming
}
// all sub enums must also derive debug
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(State)
}

fn main() {
```

Enum Composition

- Better use, use another enum!

```
    Alaska,
    // ...
    Wyoming
}
// all sub enums must also derive debug
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(State)
}

fn main() {
    let c1 = Coin::Quarter(
        State::Wyoming
    );
```

Enum Composition

- Better use, use another enum!

```
 7 // all sub enums must also derive debug
 8 #[derive(Debug)]
 9 enum Coin {
10     Penny,
11     Nickel,
12     Dime,
13     Quarter(State)
14 }
15
16 fn main() {
17     let c1 = Coin::Quarter(
18         State::Wyoming
19     );
20
21     println!("{:?}", c1);
22 }
```

Enum Composition

- Better use, use another enum!

```
enum State {
    Alabama,
    Alaska,
    // ...
    Wyoming
}
// all sub enums must also derive debug
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(State)
}

fn main() {
```

Matching with composition

- "Un-pack" by *destructuring*
- (data) is a binding to the value, within the scope

```
let information = match c1 {
    Coin::Penny | Coin::Nickel => {
        "Worthless!"
    },
    Coin::Dime => "Mostly worthless",
    Coin::Quarter(state) => {
        match state {
            State::Ohio => "Priceless",
            _ => "Worthless"
        }
    },
};
```

Matching with composition

- "Un-pack" by *destructuring*
- (data) is a binding to the value, within the scope

```
let information = match c1 {
    Coin::Penny | Coin::Nickel => {
        "Worthless!"
    },
    Coin::Dime => "Mostly worthless",
    Coin::Quarter(state) => {
        match state {
            State::Ohio => "Priceless",
            _ => "Worthless"
        }
    },
};
```

Matching with composition

- "Un-pack" by *destructuring*
- (data) is a binding to the value, within the scope

```
let information = match c1 {
    Coin::Penny | Coin::Nickel => {
        "Worthless!"
    },
    Coin::Dime => "Mostly worthless",
    Coin::Quarter(state) => {
        match state {
            State::Ohio => "Priceless",
            _ => "Worthless"
        }
    },
};
```

Matching Philosophy

- Crazy powerful language feature
- Philosophy of Rust ensures you have always handled every case
- Many implementations are non-exhaustive (Java, Python)
- "Don't produce a binary, unless the code is *provably* valid"
- Prevents a large amount of bugs at compile-time

Questions?